

Executive Summary

This report analyzes how to enrich a **read-only** Identity Operations console with Ping Identity APIs for SSO/MFA troubleshooting, dashboards and user analytics. We inventory Ping products (PingOne, PingFederate, PingAccess, PingDirectory, PingID/PingOne MFA, PingIntelligence/Protect) and identify the key read-only API endpoints. For each operational feature (auth logs, MFA status, session info, user 360, etc.) we map to specific endpoints (with HTTP methods, URLs, scopes and response fields). We propose in-memory data models (JSON schemas) and UI metrics (tables, charts) that can be computed on-the-fly from API calls (no persistent DB). Security guidance covers OAuth2 scopes (least privilege), secret management, PII masking, and audit-logging. We outline SDKs, sample code patterns, webhook/streaming options, and error-handling (pagination, rate-limit). Finally, we recommend a phased implementation plan with mermaid timeline and architecture flowchart diagrams.

Key recommendations include: use PingOne AIC's **Monitoring Logs API** for event/audit data ¹ ; use PingFederate's **Token Introspection** and **Session Management** APIs for live SSO session info ² ; use PingDirectory SCIM APIs for user profile and group lookups ³ ; use PingOne MFA APIs to enumerate enrolled devices and generate reports ⁴ ; and use PingOne Protect (AI risk) API to score user logins on demand ⁵ . All API calls require OAuth2 (PingOne: client_credentials, PingFederate: client or cert) with minimal scopes, and should handle pagination and 429 rate-limit responses with retry/backoff. In-memory caches should have TTLs (e.g. 5–15 min) to avoid repeated calls.

Security posture: enforce least-privilege roles and scopes on API clients, protect secrets (e.g. via Vault or environment vars), log tool usage to a SIEM, and design the UI so that PII (usernames, IPs) is masked or only shown to authorized roles. The app itself should be fronted by corporate SSO (PingOne, etc.) and use RBAC for internal access.

The recommended implementation steps span API design, UI/dashboards, security review, testing, and a canary rollout (see **Timeline** below). Reference architecture (mermaid diagram) shows a stateless API layer bridging the front-end to Ping services and a SIEM/log exporter.

1. Ping Identity Products & API Inventory

Product	Primary APIs	Use Cases
PingOne (AIC)	- Monitoring Logs API (GET /monitoring/logs or /tail) ¹ ⁶ - PingOne IAM: identities, apps (REST CRUD) ⁷ - PingOne MFA: devices, logs, reporting (e.g. GET /users/{id}/devices) ⁸ , POST /reports/... ⁴ - PingOne Protect (Intelligence): risk evaluations (POST /riskEvaluations) ⁵	Logs and events (auth, audit) per tenant/environment; user directory in cloud; MFA device status and logs; risk scores for fraud detection.

Product	Primary APIs	Use Cases
PingFederate	- OAuth2 / OIDC: introspection (<code>POST /as/introspect.oauth2</code>), userinfo, etc. - Session Management API (<code>GET /pf-ws/rest/sessionMgmt/users/{user}</code>) ² . - Admin (PF-WS) API: no OAuth support; uses basic auth or certs.	Token introspection to validate/inspect tokens; query/revoke user SSO sessions ² .
PingAccess	- Administrative API (<code>/pa-admin-api/v1/...</code>) with OAuth tokens from PF ⁹ (uses PF client). - No direct log API (logs to files/Splunk).	View gateway configuration or metrics; OAuth tokens from PF can call PingAccess endpoints if needed ⁹ .
PingDirectory	- SCIM 2.0 REST API (<code>/scim/v2/Users</code> , <code>/Groups</code> , etc.) ³ . - LDIF/LDAP (not applicable to a purely REST front-end).	Query user profiles, attributes and group memberships for “user 360” view.
PingID (MFA)	- If on-prem PingID is used: PingID APIs (push OTP) or use PingID admin portal. (Often replaced by PingOne MFA).	Check MFA enrollment (if legacy), device logs. (Prefer PingOne MFA instead.)
PingIntelligence / Protect	- PingOne Protect Risk API (<code>POST /riskEvaluations</code> , <code>GET /riskPredictors</code> etc.) ⁵ . No direct log API (risk decisions can be retrieved).	Provide real-time risk scores for login attempts. PingIntelligence for APIs (API security) is out of scope for user ID management.

We prioritize PingOne AIC (cloud IAM) APIs for logs, MFA, risk, as well as PingFederate (on-prem) for sessions and introspection. PingAccess and PingID are secondary or legacy in this context.

2. Endpoints for Key Features

Below we map each desired feature to specific Ping API calls. For brevity we show one endpoint per feature; in practice multiple APIs may be combined. (Full lists in next sections.)

2.1 SSO Troubleshooting Console

- **Authentication Logs:** Retrieve login events from PingOne:

`GET https://<tenant-env>/monitoring/logs?source=am-authentication&beginTime=...&endTime=...` with headers `x-api-key` / `x-api-secret`

¹ . This returns log entries (payload JSON with timestamp, source, etc.) ⁶ . Use pagination (pagedResultsCookie) or `/tail` for live feed ¹⁰ .

- **PingFederate Token Introspection:**

POST `https://<pf-domain>/as/introspect.oauth2` (form data: `token`, `token_type_hint`) with Basic or Bearer auth. Returns JSON: `{active, client_id, scope, exp, username...}`. Shows if a token is valid and contains user info.

- **PingFederate Active Sessions:**

GET `https://<pf-domain>/pf-ws/rest/sessionMgmt/users/{user_key}` with OAuth client auth. Returns all active sessions for the user: includes fields like `sri`, `status`, `lastActivityTime`, array of `authnSessions` (with `id`, `creationTime`, timeouts, authn source, context IP and user-agent) ².

- **PingAccess Logs:** PingAccess does not expose request logs via API. Instead, export logs via a SIEM (e.g., Splunk). The PingAccess admin API (requires PF OAuth token) can return configuration/status but not request logs.

- **Token Status Dashboard:** Aggregate results from introspection/sessions to show active sessions per user. E.g. chart of sessions per application, or table of token details.

2.2 MFA Diagnostics

- **MFA Enrollment & Devices:**

GET `https://api.pingone.com/v1/environments/{envID}/users/{userID}/devices` returns all MFA devices (mobile, SMS, OATH, FIDO) for a user. Each device has `type`, `status`, etc. ¹¹.

- **MFA Device Logs:**

For native (mobile) devices, we can **request** device logs by `PUT /environments/{envID}/users/{userID}/devices/{deviceID}/logs` with body `{"status":"PENDING"}` ¹². The user's device will upload logs (on request) within 24h. After the device sends logs, you can retrieve them via `GET /.../logs` (same URL) to read the log contents.

- **MFA Reports:**

PingOne MFA API provides reporting endpoints ⁴: e.g. `POST /reports/smsDevices` returns all users with SMS devices, `POST /reports/mfaEnabledDevices` generates a CSV file of all users with any MFA. Poll results via `GET /reports/{id}/result`. Use these to compute, e.g., percent of users with MFA. (See Table below.)

2.3 Leadership Dashboards / Analytics

- **Overall Usage Metrics:**

Use PingOne Logs API with filters (source like `am-authentication`) to compute totals: e.g. total logins, failures, success rate over time. Graph trends.

- **MFA Adoption:**

Use MFA Reports (above) to calculate MFA enablement: e.g.

$$\text{MFA_enablement} = (\text{count of users with at least one enrolled MFA device}) / (\text{total active users}) * 100\%$$

Data from GET `/devices` for each user (or aggregate from report).

- **Geolocation and Risk Stats:**

If log payloads include IP/country, aggregate logins by region. Also, call

POST `/risk-evaluations` for each login attempt (or periodically) to score risk; show histogram of risk levels.

- **System Health:**

Ping products do not expose real-time health via these APIs. Recommend separate monitoring (e.g. PingAccess node status via GET `/pa-admin-api/v1/health` with OAuth, PingFederate system checks, etc.). At least check for API response delays.

2.4 User “360” Profile

- **User Attributes & Groups:**

PingDirectory SCIM: GET `/scim/v2/Users?filter=username eq "{username}"` or GET `/scim/v2/Users/{id}` ³. Returns full profile (email, name, attributes). Similarly, GET `/scim/v2/Groups?members eq "{userID}"` to list groups.

- **Active Sessions:**

Use PingFederate Session API for that user ².

- **Enrolled Devices:**

Use PingOne MFA API GET `/environments/{env}/users/{user}/devices` ¹¹.

- **Auth History:**

Query PingOne logs for entries with that user (filter on `payload` if JSON contains username or use `transactionId` chaining) ¹³.

- **MFA Status & Settings:**

From MFA device list and PingOne MFA settings API: GET `/environments/{env}/mfa/settings` returns global MFA config, and GET `/environments/{env}/users/{user}/devices`.

- **Visualization:** Present combined data in UI: user card with ID/name/email, list of devices, group list, recent login events.

2.5 Compliance / Audit View

- **Audit Log Retrieval:**

All admin actions and system events in PingOne are logged. Use same `/monitoring/logs` endpoint to retrieve admin audit events (source parameter like `audit` or `idm-admin-audit`). For

PingFederate, its audit log can be exported to SIEM and polled; PingFederate has no REST audit endpoint.

- **Data Model:** Maintain in-memory buffer of recent audit events. Schema example (JSON):

```
{
  "timestamp": "...",
  "actor": "admin@example.com",
  "action": "UpdatePolicy",
  "target": "RiskPolicySet",
  "result": "SUCCESS",
  "ipAddress": "1.2.3.4"
}
```

- **UI:** Table of audit events; filters by time, admin user, action type. Summaries (counts per admin, per action).

2.6 Geo / Risk Monitoring

- **Geo:** The log payload for authentication often includes IP/Geo. Use log entries (`monitoring/logs`) and extract `payload.ipAddress` or similar. Compute chart of logins by country.
- **Risk:** For each login (or on-demand), call PingOne Protect: `POST /risk-evaluations` with context (user ID, IP, device info) ⁵ . The JSON response contains a risk score or decision (e.g. "ALLOW" or "DENY") with details. Use this to highlight anomalous logins.

2.7 Troubleshooting Toolkit

- Provide quick lookups: given a user or token, call introspect, session list, device list, etc. E.g., allow entering a `transactionId` to fetch specific logs via `/monitoring/logs?transactionId={id}` ¹³ . Show flow of that authentication.

3. Endpoint Reference Tables

Below are tables mapping key endpoints to their usage, required scopes, and sample fields. (Scopes are examples; actual scopes depend on your Ping role setup.)

Authentication & Session Endpoints

Feature	Endpoint (Method)	Description	Auth/Scope	Key Response Fields
PingOne Logs (AIC)	GET /monitoring/logs?source={src}&... ¹	Retrieve log events (last 24h or specify time range)	HTTP headers x-api-key / x-api-secret (API key auth) ¹ . Scope: e.g. logs.read.	JSON result[]: each entry has payload (event JSON or text), timestamp, type, source ⁶
PingOne Logs (tail)	GET /monitoring/logs/tail?source={src}	Tail new log entries since last call	as above	Same as above; use pagedResultsCookie to continue.
PF Introspect	POST /as/introspect.oauth2	Validate/access details of an access token	Client auth (Basic or Bearer). Scopes: none (token itself must be valid).	active, client_id, scope, exp, user info (username, custom claims).
PF Session Query	GET /pf-ws/rest/sessionMgmt/users/{user_key} ²	List all live sessions for user_key (unique ID)	OAuth client (client_credentials with SessionMgmt scope). Requires PingFed client config ¹⁴ .	JSON object: status, lastActivityTime, authnSessions[] (with each id, creationTime, idleTimeout, authnSource, etc.), contextData (IP, user-agent) ² .

MFA Endpoints

Feature	Endpoint (Method)	Description	Auth/Scope	Key Fields
List Devices	GET /environments/{env}/users/{user}/devices ¹¹	List all MFA devices for a user	Bearer token from PingOne (client_credentials). Scope: mfa.devices.read.	JSON array of devices: each has id, type (e.g. MOBILE / SMS), status (ACTIVE/PENDING) ¹¹ .
Request Device Logs	PUT /.../devices/{deviceID}/logs ¹⁵	Ask device to upload logs (for native devices)	Bearer token. Scope: mfa.devices.write.	Sends back JSON with status: PENDING; system updates when logs received.

Feature	Endpoint (Method)	Description	Auth/Scope	Key Fields
Get Device Logs	GET /.../devices/{deviceId}/logs	Retrieve logs sent by device (after PENDING)	Bearer token. Scope: mfa.devices.read.	Raw log text or JSON payload of device events.
Report SMS Devices	POST /environments/{env}/reports/smsDevices ⁴	Generate report of all SMS MFA devices (immediate JSON response)	Bearer token. Scope: mfa.reports.read.	Returns reportResults: array of {userID, userName, deviceId, ...} entries.
Report MFA-enabled	POST /environments/{env}/reports/mfa-enabled ⁴	Generate report of all users with any MFA device (CSV file)	Bearer token. Scope: mfa.reports.read.	Returns job ID; poll GET /reports/{id}/result: response includes URL for CSV or inline rows.
Report Results	GET /environments/{env}/reports/{reportID}/result	Fetch results of report (JSON or CSV)	Bearer token. Scope: mfa.reports.read.	Job status, and either entries array or file download.

SCIM (Directory) Endpoints

Feature	Endpoint (Method)	Description	Auth/Scope	Key Fields
Search Users	GET /scim/v2/Users?filter=username eq "{name}" ³	Find user by username (or other filter)	OAuth token from PingFederate (client_credentials) with SCIM scope.	Resources[]: each resource is a user with attributes (id, userName, name, emails, etc.).
Get User by ID	GET /scim/v2/Users/{id} ³	Retrieve single user entry	as above	User JSON with userName, emails, memberships, etc.
Search Groups	GET /scim/v2/Groups?filter=members eq "{userDN}"	List groups containing user	as above	Resources[] with each group's displayName, members array.

Feature	Endpoint (Method)	Description	Auth/Scope	Key Fields
Get Group by ID	GET /scim/v2/Groups/{id}	Get group details	as above	Group JSON with <code>displayName</code> , <code>members</code> .

Note: To use SCIM, configure PingFederate as OAuth provider for PingDirectory (see PingDirectory SCIM guide). Scopes like `scim.read` may be required. The SCIM API respects ACIs so only allowed attributes are returned.

Risk API (PingOne Protect)

Feature	Endpoint (Method)	Description	Auth/Scope	Key Fields
Compute Risk	POST /environments/{env}/risk-evaluations ⁵	Evaluate risk for a user login context	Bearer token (PingOne Protect client). Scope: <code>risk.evaluate</code> .	Response JSON: <code>decision</code> (ALLOW/DENY/CHALLENGE), <code>score</code> (0–1), <code>policyId</code> , <code>metadata</code> (details).

Additional Endpoints

- **PingAccess Admin:** GET /pa-admin-api/v1/health or other admin calls. Auth via PF OAuth token with scope configured ⁹. (No logs API.)
- **PingID (Legacy):** If still used, PingID REST API can query user status, but it's deprecated in favor of PingOne MFA.
- **Webhook/Streaming:** PingOne supports pushing events to external SIEM. (Docs: “Monitoring activity with Splunk” ¹⁶, webhook connectors ¹⁷.) Alternatively, you **poll** /monitoring/logs in a loop or tail.

4. Data Models and In-Memory Objects

Since the app is read-only, define JSON schemas to hold aggregated data temporarily (cache with TTL). Example models:

```
// Model for a user's 360 profile:
{
  "userId": "1234-abcd",
  "userName": "alice@example.com",
  "displayName": "Alice Smith",
  "emails": ["alice@example.com"],
  "phoneNumbers": ["+1-555-1234"],
  "groups": ["Engineering", "VPN Users"],
  "mfaDevices": [
    {"id": "dev1", "type": "TOTP", "status": "ACTIVE"},
  ]
}
```

```

    {"id": "dev2", "type": "MOBILE_PUSH", "status": "PENDING"}
  ],
  "sessions": [
    {
      "id": "abcd1234",
      "creationTime": "2023-04-20T10:00:00Z",
      "idleTimeout": "2023-04-20T10:30:00Z",
      "lastActivityTime": "2023-04-20T10:15:00Z",
      "ipAddress": "1.2.3.4",
      "userAgent": "Mozilla/5.0",
      "authnSource": "HtmlFormAdapter"
    }
  ],
  "lastLogin": "2023-04-20T10:15:00Z",
  "status": "ACTIVE"
}

```

```

// Model for aggregated metrics (cached):
{
  "totalLogins": 1245,
  "successRate": 0.987,
  "failedLogins": {
    "invalid_password": 5,
    "account_locked": 3
  },
  "mfaEnabledPercent": 76.5,
  "activeSessions": 53,
  "topRiskScores": [0.95, 0.87, 0.72]
}

```

- **Retention:** Cache these objects in memory for short TTL (e.g. 5–15 minutes). Recompute via APIs after expiration or if user refreshes view.
- **Pagination:** For endpoints returning many items (logs, sessions), implement loops. E.g., for logs:

```

cursor = None
while True:
    params = {"source": "am-authentication", "beginTime": T1, "endTime":
T2}
    if cursor: params['_pagedResultsCookie'] = cursor
    resp = requests.get(..., headers=headers, params=params).json()
    process(resp['result'])
    cursor = resp.get('pagedResultsCookie')
    if not cursor: break

```

- **Backoff:** On HTTP 429 or timeout, retry with exponential backoff.

5. Sample API Calls and Snippets

PingOne Logs (Python):

```
import requests, time
API_KEY = os.environ["PINGONE_API_KEY"]
API_SECRET = os.environ["PINGONE_API_SECRET"]
TENANT_FQDN = "api.pingone.com" # or region
headers = {"x-api-key": API_KEY, "x-api-secret": API_SECRET}
params = {"source": "am-authentication", "beginTime": "2026-04-21T00:00:00Z"}
url = f"https://{TENANT_FQDN}/monitoring/logs"
resp = requests.get(url, headers=headers, params=params)
logs = resp.json().get("result", [])
for entry in logs:
    print(entry["timestamp"], entry["payload"][:100])
```

PingFederate Introspect (cURL):

```
curl -X POST "https://pf.example.com/as/introspect.oauth2" \
-u "pf-client:secret" \
-d "token=eyJhbGciOi..." -d "token_type_hint=access_token"
```

Response (JSON):

```
{
  "active": true,
  "client_id": "pf-client",
  "exp": 1713648000,
  "scope": "openid profile email",
  "username": "alice@example.com"
}
```

PingFederate Session API (Node.js):

```
const axios = require('axios');
const pfClientToken = 'Bearer ' + token; // obtained via client_credentials
OAuth
const userKey = encodeURIComponent('alice@example.com');
axios.get(`https://pf.example.com/pf-ws/rest/sessionMgmt/users/${userKey}`, {
  headers: { Authorization: pfClientToken }
}).then(res => {
```

```
console.log("Active sessions:", res.data.authnSessions);
});
```

PingOne MFA Device List (Python):

```
env_id = "env-1234"
user_id = "user-5678"
url = f"https://api.pingone.com/v1/environments/{env_id}/users/{user_id}/
devices"
resp = requests.get(url, headers={"Authorization": f"Bearer {token}"})
devices = resp.json()[ "_embedded" ][ "devices" ]
print("MFA devices:", [d["type"] for d in devices])
```

PingOne Create Risk Evaluation (Python):

```
url = f"https://api.pingone.com/v1/environments/{env_id}/riskEvaluations"
payload = {"user": {"identifier": user_id, "ipAddress": "1.2.3.4"}}
resp = requests.post(url, json=payload, headers={"Authorization": f"Bearer
{token}"})
risk_result = resp.json()
print("Risk decision:", risk_result["decision"], "score:", risk_result["score"])
```

All secrets (client IDs, API keys) must come from secured config (env vars or Vault). Do not hard-code credentials in code. Use TLS (HTTPS) and rotate tokens regularly.

6. UI Metrics and Dashboards

Suggested Visualizations:

- **Auth Events Table:** Columns: Timestamp, User, Application, Result (Success/Failure), IP, Auth Method. This shows recent login attempts for ops teams.
- **Login Success Rate Chart:** Line chart of successful vs. failed logins over time (e.g. last 24h, 7d).
- **MFA Enablement Gauge:** Pie or donut showing % users with MFA enabled vs total (computed via `/reports/mfa-enabled`).
- **Top Failure Reasons:** Bar chart of most common auth failure codes from logs (e.g., invalid password, OTP timeout).
- **Active Sessions:** Numeric summary of current sessions (PingFed Session API) and list of sessions (user, start time).
- **Risk Level Distribution:** Histogram of user logins by risk score (PingOne Protect).
- **User 360 Panel:** Upon selecting a user, show a card with basic info, groups, devices, sessions, recent logins, and MFA status. Possibly a mini-timeline of that user's authentication events.

Data for charts should be computed in-memory from API results (filter/aggregate on JSON payloads). For example, to get success rate, count `payload.success==true` vs `false` in auth logs. Use chart libraries (Chart.js, D3, etc.) with the data arrays.

7. Troubleshooting Workflows

- **Single User Troubleshoot:** Operator searches user → fetch user profile (SCIM), fetch active sessions (PingFed), fetch devices (PingOne), fetch recent logs (PingOne monitoring) → display collated info.
- **Token Issue:** Paste token in UI → call introspect or PingFed `/introspect` → show validity, scopes, expiry, associated user.
- **Failed Login Spike:** Show chart of login attempts; on spike click, filter logs for that interval to identify cause (e.g. correct vs wrong password, MFA challenges).
- **MFA Enrollment Check:** Select user → show if they have any devices; if none, prompt enrollment. Admin console could be opened from here.
- **Risk Alert:** Show user with high-risk login → view the risk evaluation details, compare with policy.

Each workflow is powered by API calls above and visual analysis.

8. Available SDKs & Sample Repositories

- **PingOne SDKs:** For user flows (OIDC login), Ping provides [JS/Java/Swift/etc. SDKs](#) (e.g. `@ping-identity/p14c-js`) and sample apps ¹⁸. But for administrative APIs, no official JS/Python, so use standard HTTP libraries.
- **PingOne AIC Go SDK:** (PingOne AIC API) [Go client](#) exists for PingOne AIC (<https://github.com/pingidentity/PingOne-AIC-Go>). Example usage for identity operations.
- **PingFederate SDK:** PingFederate has a [Go SDK](#) for its administrative REST API. Could use it for session API calls.
- **PingDirectory SCIM:** No official SDK; use any SCIM client or HTTP.
- **Python/Node:** Use `requests` (Python) or `axios` (Node) for generic calls. See snippets above.
- **Rate-Limit Handling:** Use `Retry-After` header or known QPS limits. Eg. PingOne docs recommend throttling to avoid 429 ¹⁰. Implement exponential backoff.
- **Secrets:** Use environment variables or a secrets vault. E.g. in Kubernetes, use a Secret for the PingOne API key and mount as env var.

9. Webhooks & Event Streaming

- **PingOne Webhooks:** PingOne DaVinci (Flow) can push events to webhooks ¹⁷, but the Monitoring Logs API is simplest for pulling events.
- **Log Streaming:** PingOne AIC supports streaming logs to external SIEM (e.g. Splunk) ¹⁷. For near-real-time UI updates, you could tail the `/monitoring/logs/tail` endpoint ¹⁹ continuously or use PingOne's webhook connector to send events to your backend.
- **PingAccess:** Can integrate with SIEM via syslog, but no direct webhook.
- **Alternative Polling:** If no push, schedule periodic polls (e.g. every minute) to update dashboards.

10. Security Considerations

- **Least Privilege:** Grant the API client only necessary scopes. For example, a “viewer” role in PingOne with only `monitoring.read` and `mfa.read` scopes for this app. Map app roles to PingOne admin roles.
- **RBAC:** Integrate the console with corporate SSO; only allow Cyber/Identity team users. Internally, separate roles (e.g. “auditor” vs. “admin”) to control what data is visible (e.g. hide PII from auditors).
- **PII Masking:** Redact sensitive fields in logs (last 4 of account numbers, mask emails if needed). Use PingOne’s log filtering (configure policies to exclude PII) or mask in UI.
- **Token Management:** Use short-lived client credentials if possible. Rotate secrets regularly. Store no tokens in UI.
- **Audit Logging:** Log every UI action (searches, views, exports) to your SIEM. Include userID of admin, timestamp. Also log all API errors.
- **Network Security:** Call Ping APIs only over HTTPS. If self-hosted console, keep it inside corporate VPN or use a secure cloud account.
- **Zero-Trust:** The console itself should require multi-factor login. Consider network isolation (access only from Cyber team’s subnet).

11. Testing Plan

- **Unit Tests:** Mock API clients; verify JSON parsing and data aggregation logic.
- **Integration Tests:** Against a PingOne **test environment** and a PingFederate **test instance**. Use known test data to validate endpoints return expected fields.
- **Security Tests:** Ensure no secret leakage (lint config), check CORS if front-end, pen-test UI for XSS (even though read-only), and ensure TLS. Use static code analysis tools.
- **Performance:** Simulate heavy log polling (e.g. many users, many events). Ensure UI still responsive. Monitor API rate-limits.
- **Edge Cases:** Token expiry, unreachable APIs. The app should handle 401 (re-auth) and 503/429 (retry) gracefully.

12. Rollout Strategy & Timeline

We recommend a phased rollout:

1. **Proof of Concept (2–3 wks):** Build core API layer calling PingOne logs and PingFederate introspect. Simple web UI to display a few metrics. Test in dev Ping environments.
2. **Feature Build (4–6 wks):** Add remaining features (MFA, user 360, dashboards). Implement caching and pagination. Internal code review and security review (address compliance requirements).
3. **Pilot Release (2 wks):** Deploy to a subset of Cyber team with limited data. Feature-flag new dashboard. Collect feedback.
4. **Expanded Test (2 wks):** Roll out to full Cyber team. Conduct training, update docs.
5. **Production Launch (1 wk):** Enable for leadership (read-only view), monitor usage.
6. **Iteration & Hardening (ongoing):** Based on user feedback, add enhancements (e.g. new charts, filtering). Continuously review logs and performance.

Below is a high-level timeline diagram (mermaid gantt style):

```
gantt
    title Implementation Timeline
    dateFormat YYYY-MM-DD
    section Setup
    Planning & Design      :crit, a1, 2026-04-22, 2026-05-05
    Environment Prep (test) :a2, after a1, 2w
    section Development
    Core API Integration   :a3, after a2, 3w
    UI / Dashboard Build   :a4, after a3, 4w
    Security Implementation :a5, after a4, 2w
    section Testing
    Unit & Integration Tests :a6, after a5, 3w
    Performance Testing    :a7, after a6, 1w
    section Rollout
    Internal Pilot         :a8, after a7, 2w
    Full Deployment        :a9, after a8, 1w
    Post-Deployment Review :a10, after a9, 1w
```

13. Architecture Diagram (Mermaid)

```
flowchart LR
    UI[Web Frontend (React/Angular)] -->|API calls| APIGW[Stateless API Layer]
    APIGW --> PingOne[PingOne AIC APIs]
    APIGW --> PingFed[PingFederate APIs]
    APIGW --> PingDir[PingDirectory (SCIM API)]
    APIGW --> SIEM[SIEM/Log System]
    PingOne -->|Logs/Events| SIEM
    PingFed -->|Audit Logs| SIEM
    UI -. AD[Corporate SSO (PingOne or AD)]
    AD --> UI
    SIEM --> APIGW[Pull logs via API (optional)]
```

Figure: Data flow – User queries come through the UI to an API backend, which calls PingOne (logs, MFA, risk), PingFederate (token/session), PingDirectory (user info). Key events are also sent to a SIEM (and can be polled by the app). The UI itself is protected by corporate SSO and only reads data.

(For slide export: This mermaid diagram can be rendered to SVG/PNG. Ensure legible fonts and adjust sizes as needed.)

14. Conclusion and Next Steps

By leveraging Ping Identity's rich API surfaces, the cyber team's read-only console can provide deep visibility into SSO/MFA operations without building custom infrastructure. The strategy is to **pull** from Ping's logging, session, and MFA endpoints, aggregate in-memory, and visualize in dashboards. Key next steps:

- **Prototype** the log retrieval and introspection flows to verify API access and data formats.
- **Design** the UI mockups (tables, charts) aligned with user needs.
- **Define** OAuth2 roles/scopes in PingOne and PingFederate for the app.
- **Implement** caching strategies (e.g. Redis or local memory) with expiration.
- **Plan** secure storage for secrets (use secrets manager, not in code).
- **Coordinate** with Ping Identity support for any advanced features (e.g. enabling SCIM, API rate-limit increases).
- **Review** IGA policies (audit logs retention, least privilege, data masking requirements).

With these integrations, the operations team can diagnose incidents faster, track security metrics, and demonstrate compliance (MFA coverage, audit trails) – all within a user-friendly console.

1 6 10 13 19 Retrieve log entries using REST API | PingOne Advanced Identity Cloud

<https://docs.pingidentity.com/pingoneaic/tenants/audit-debug-logs-pull.html>

2 14 Session Management API by user identifiers | PingFederate Server

https://docs.pingidentity.com/pingfederate/13.0/developers_reference_guide/pf_sess_manage_api_user_identif.html

3 PingDirectory and PingDirectoryProxy SCIM 2.0 API Reference | PingDirectory and PingDirectoryProxy SCIM 2.0 API Reference

<https://developer.pingidentity.com/pingdirectory/directory-proxy-scim/introduction.html>

4 8 11 12 15 Send MFA Device Logs | PingOne Platform APIs

<https://developer.pingidentity.com/pingone-api/mfa/users/mfa-devices/send-mfa-device-logs.html>

5 Create Risk Evaluation | PingOne Platform APIs

<https://developer.pingidentity.com/pingone-api/protect/risk-evaluations/create-risk-evaluation.html>

7 PingOne Advanced Identity Cloud API | PingOne Advanced Identity Cloud APIs

<https://developer.pingidentity.com/pingoneaic-api/index.html>

9 PingFederate and PingAccess REST APIs using Authorization token - Stack Overflow

<https://stackoverflow.com/questions/40547068/pingfederate-and-pingaccess-rest-apis-using-authorization-token>

16 17 PingOne Services logging and reporting | PingOne

https://docs.pingidentity.com/pingone/getting_started_with_pingone/p1_services_logging_reporting_overview.html

18 GitHub - pingidentity/pingone-sample-js: PingOne for Customers Sample App - OIDC Authentication Quick Start. Sample code providing a quick and simple introduction to PingOne for Customers using simple JavaScript or Python including code that can be modified for use in your own applications. · GitHub

<https://github.com/pingidentity/pingone-sample-js>